Humanoid Robot NLP Module

Intern Task Guide & Deliverable Specification

Project	Unitree G1 Humanoid Office Assistant
Module	NLP Pipeline (ASR → Intent → Response → TTS)
Duration	
Hardware target	Unitree G1 EDU — NVIDIA Jetson Orin NX (100 TOPS, 16 GB)

> Purpose of this document: This guide gives the intern everything they need to start, execute, and complete their assigned tasks independently. Each task section explains the goal in plain language, lays out a day-by-day plan, specifies exact deliverables, and lists success criteria. When in doubt, re-read the goal statement at the top of each section — it anchors every decision you make.

Section 0 — Orientation

Before diving into the three tasks, spend your time on building context. You cannot write good training data without understanding what the robot needs to do. You cannot benchmark models without understanding the hardware constraints. This orientation block is not optional — it will save you from going down wrong paths later.

First step

1. Understand and learn about ANN, LLM, Parameters, Self Attention, Fine Tuning, Quantization, RAG, Transfer learning strategies, Parameter-efficient fine-tuning (LoRA, adapters), Hyperparameter tuning, Logging tools (e.g., Weights & Biases), Docker basics, GPU usage (CUDA basics), Git & GitHub workflows
2. Study the NLP architecture diagram. Trace the path: a person speaks → ASR → intent classification → dialogue manager → response generation → TTS → robot speaks. Make sure you can explain every step.
3. Set up your development environment: Python 3.11, a virtual environment, and install the baseline packages listed in Appendix A.

Second step

Familiarise yourself with the Jetson Orin hardware if available. Confirm SSH access, check available RAM (jetson_stats), and understand why the 16 GB memory ceiling shapes every model choice in this project.

Third step

Read the MCP (Model Context Protocol) documentation overview. Understand what a "tool" is in this context — a function the LLM can call to retrieve external information or trigger an action.

Write a one-page orientation summary covering: what the robot does, how the NLP pipeline works, and what your three tasks contribute. Share the doc for a quick alignment check. This is not graded — it is a communication checkpoint.

> Key mindset: You are building infrastructure that the robot will use to talk to real people in a real office. Every shortcut you take in data quality or benchmarking will show up as a confused robot in front of a visitor. Treat your outputs as production inputs.

Orientation deliverable

Deliverable	Format	Must include	How to submit
Orientation summary	1-page PDF or doc	Robot capabilities, NLP pipeline overview, your task plan in your own words	Share the document

Task 1 Data Curation & Annotation

Build the fine-tuning dataset for the intent classifier and NER model

Why this task matters

The intent classifier and named entity recogniser (NER) are the two models that sit between Whisper output and the dialogue manager. Their quality directly determines whether the robot understands what a person wants. A model fine-tuned on 500+ real, annotated, office-domain utterances will outperform a generic zero-shot model by a significant margin — not because fine-tuning is magic, but because the model has seen the specific vocabulary, room names, person names, and phrasing patterns that occur in this environment.

Think of this task as building the "vocabulary" the robot needs to function well. If someone says "Can you take me to the MD's cabin?", a generic model might miss that "cabin" is a room synonym in some Indian office contexts, or that "MD" is an entity (a person's title). Your annotations teach the model these patterns.

Detailed task plan

Week	Focus	Daily breakdown
Days	Utterance collection	Generate 300 utterances manually and via GPT-4o. Cover all five SOW capability areas. Aim for variety in phrasing — "Take me to Room B", "Can you show me where Room B is?", and "I need to get to Room B" are three different phrasings of the same intent.
Days	Long-tail synthesis	Use GPT-4o to generate 200+ additional utterances for rare cases: visitor names, specific room codes, multilingual requests (Telugu, Hindi). Verify each synthesised utterance sounds natural — reject any that feel robotic or overly formal.
Days	Annotation sprint	Use Label Studio to annotate all utterances with intents and entities. Annotate in batches of 50. After each batch, review for consistency — the same phrase should always get the same intent label.
Days	QA and export	Run inter-annotator agreement check (Cohen's Kappa if a second annotator is available). Export to JSON in the format specified below. Write the data card document.

Intent taxonomy (use exactly these labels)

Use the following intent labels. Do not invent new ones — if an utterance does not fit, mark it as "out_of_scope" and note it in your log.

Intent label	Meaning	Example utterances
greet	User is greeting the robot or starting a conversation	"Hello", "Hi there", "Good morning"
navigate_to	User wants to go to a location	"Take me to Room 4B", "Where is the cafeteria?"
escort_guest	User wants robot to escort someone else	"Please take this visitor to HR"
query_schedule	User asking about meetings or calendar	"What meetings do I have today?"
query_info	User asking a factual question	"Who is the CEO?", "What floor is Finance on?"
set_reminder	User wants a reminder set	"Remind me about the 3pm call"
deliver_item	User wants robot to carry something	"Bring this document to Mr. Sharma"

room_setup	Meeting room preparation request	"Turn on the screen in Room 3"
out_of_scope	Request the robot cannot handle	"Can you order food?", "What is the weather in Tokyo?"

Entity types to annotate

- PERSON — Any person's name or title used as a reference. Examples: "Mr. Kumar", "the MD", "Dr. Priya".
- ROOM — Room names, numbers, or area names. Examples: "Room 4B", "the boardroom", "HR cabin", "reception".
- TIME — Any time or date expression. Examples: "3pm", "tomorrow morning", "in 10 minutes".
- ITEM — Physical objects to be carried or delivered. Examples: "this document", "the visitor badge", "a water bottle".
- ORG_UNIT — Departments or organisational units. Examples: "HR", "Finance", "the C-suite floor".

GPT-4o synthesis prompt template

Use the following prompt structure when generating synthetic utterances. Keep a record of every prompt you used and how many utterances each prompt produced.

```
You are generating training data for an office humanoid robot's intent
classifier. Generate 20 natural utterances for the intent "navigate_to". Each
utterance should: (1) sound like something a real office worker or visitor would
say, (2) vary in length and formality, (3) include a variety of room names from
this list: [Room 4B, Boardroom, HR Cabin, Cafeteria, Reception, Server Room].
Output as a JSON array of strings. Do not include any explanation or preamble.
```

> Important: Always review synthesised utterances for naturalness before adding them to the dataset. GPT-4o occasionally generates phrasing that sounds translated or unnatural in Indian English office contexts. If you would not say it yourself, do not keep it.

Required output format (JSON)

Every annotated utterance must be exported in this format. Do not change field names — downstream code depends on them.

```
{  "id": "utt_001",  "text": "Please take me to the boardroom",  "language": "en",
"source": "manual",  "intent": "navigate_to",  "entities": [    { "text":
"boardroom", "label": "ROOM", "start": 24, "end": 33 }  ],  "annotator":
"intern_name",  "confidence": "high"}
```

"source" field: use "manual" for utterances you wrote yourself, "synthesised" for GPT-4o outputs, and "collected" for utterances gathered from real people.

Success metrics — Task 1

Metric	Minimum target	Stretch target	Why it matters
Total annotated utterances	500	700+	Below 500, fine-tuning will underfit on rare intents
Intent coverage	All 9 intents represented	All intents have 50+ examples	Imbalanced data causes the model to ignore rare classes
Entity annotation completeness	All 5 entity types present	Each type has 80+ examples	NER model needs sufficient examples per entity type
Language distribution	80% English, 10% Hindi, 10% Telugu	Each language has 80+ utterances	Robot must handle all three languages
Synthesised vs manual ratio	Max 40% synthesised	Max 30% synthesised	Too much synthetic data degrades real-world performance

Deliverables — Task 1

Deliverable	Format	Must include	How to submit
Annotated dataset	JSONL file	500+ utterances in the specified JSON format, one per line	Commit to /data/utterances/ in shared repo
Data card	Markdown doc	Total count, per-intent counts, per-language counts, annotation process, known gaps	Commit to /data/DATA_CARD.md
Synthesis log	CSV	Each GPT-4o prompt used, how many outputs were kept vs rejected, rejection reasons	Commit to /data/synthesis_log.csv
Label Studio config	XML export	The labelling interface config so anyone can reproduce the annotation setup	Commit to /data/label_studio_config.xml

Task 2 Model Benchmarking & Selection

Find the best-performing LLM for the Jetson Orin NX using Ollama

Why this task matters

The Jetson Orin NX has 16 GB of RAM shared between the CPU and GPU. This is not a lot — Whisper large-v3 alone uses ~3 GB. Everything the LLM needs (model weights, KV cache, active context) must fit in what remains. Choosing a model that is too large means the robot runs out of memory mid-conversation and crashes. Choosing one that is too slow means the human waits 8 seconds for a response and the interaction feels broken.

This task is about finding the model that hits the sweet spot: fast enough to feel natural (under 2 seconds to first spoken word), accurate enough to handle office conversations, and small enough to run stably alongside the rest of the NLP pipeline.

Detailed task plan

Week	Focus	Daily breakdown
Days	Ollama setup on Jetson	Install Ollama on the Jetson Orin NX. Verify ARM64 compatibility. Pull the three candidate models. Run a basic sanity check: ask each model "What is the meeting schedule?" and confirm it responds without crashing.
Days	Latency benchmarking	Write the benchmarking script (template provided below). Run each model against the 50-utterance test set. Record TTFT (time to first token) and TPS (tokens per second) for each. Run each test 3 times and take the median — single runs are noisy.
Days	Memory profiling	Use jetson_stats (jtop) to record RAM usage during inference. Record peak RAM, steady-state RAM, and whether any model triggers memory swapping. A model that fits in RAM but causes swapping is effectively too large.
Days	Quality evaluation	Run each model against 30 office-scenario prompts from your Task 1 dataset. Grade responses on a 1–3 scale: 1 = wrong or hallucinated, 2 = acceptable, 3 = correct and natural. Total the scores. This is your quality signal.
Days	Report and recommendation	Compile all numbers into the report template. Write a clear recommendation paragraph: which model to use, why, and what the trade-offs are. The NLP lead will use this recommendation to make the final architecture decision.

Models to benchmark

Model	Parameters	Ollama tag	Est. RAM usage	Hypothesis
Llama 3.2	1B	llama3.2:1b	~1.5 GB	Fastest; quality may be too low for complex queries
Llama 3.2	3B	llama3.2:3b	~2.8 GB	Best balance candidate — our primary hypothesis
Mistral v0.3	7B	mistral:7b	~5.5 GB	Highest quality; may be too slow or RAM-heavy
Phi-3 Mini	3.8B	phi3:mini	~3.2 GB	Microsoft's edge-optimised model — worth comparing

Benchmarking script structure

Write your benchmarking script in Python. It should do the following, in order: load the test utterances from a file, send each utterance to the model via the Ollama Python SDK, record TTFT (time from request send to first token received) and total generation time, compute TPS as total_tokens / total_time, and write results to a CSV. The structure below shows the key measurement points.

```
import ollama, time, csvdef benchmark_model(model_name, prompts):    results = []    for
prompt in prompts:        start = time.perf_counter()        first_token_time = None
token_count = 0        # Stream the response to capture TTFT accurately        for chunk
in ollama.chat(model=model_name,
messages=[{"role":"user","content":prompt}],
stream=True):            if first_token_time is None:                first_token_time =
time.perf_counter() - start            token_count += 1        total_time =
time.perf_counter() - start        tps = token_count / total_time        results.append({
"model": model_name,            "prompt": prompt,            "ttft_ms":
round(first_token_time * 1000),            "total_ms": round(total_time * 1000),
"tokens": token_count,            "tps": round(tps, 1)        })    return results
```

Test prompt set (use these exact 10 as your baseline)

Use these 10 prompts as your fixed comparison set. They cover the main intent categories and vary in complexity. Run all models against the same set.

- "Take me to Meeting Room 4B."
- "Where is the HR department?"
- "What meetings does the CEO have this afternoon?"
- "Please escort this visitor to the reception area."

- "Remind me about the 3pm board call."
- "Who is visiting us from TechCorp today?"
- "Can you carry this document to Mr. Sharma's office?"
- "Turn on the projector in the boardroom."
- "Mujhe HR office le chalo." (Hindi: Take me to HR office)
- "Nannu reception ki teesukello." (Telugu: Take me to reception)

Success metrics — Task 2

Metric	Minimum target	Stretch target	Why it matters
TTFT (median, p95)	p50 < 600ms, p95 < 1200ms	p50 < 350ms, p95 < 700ms	TTFT drives perceived responsiveness — human feels the pause before any words come out
TPS (tokens per second)	> 15 TPS	> 30 TPS	Below 15 TPS, streaming speech sounds choppy and unnatural
Peak RAM during inference	< 6 GB (leaves room for Whisper)	< 4 GB	Total pipeline must fit in 16 GB alongside OS and other processes
Quality score (out of 90)	> 65 / 90	> 78 / 90	Below 65, response quality is insufficient for real users
Memory swapping events	Zero swap events during test	Zero swap events	Any swapping causes multi-second freezes that break conversation

Deliverables — Task 2

Deliverable	Format	Must include	How to submit
Benchmark results CSV	CSV	Per-model, per-prompt: TTFT, TPS, total time, token count	Commit to /benchmarks/results.csv
Memory profile logs	CSV + screenshots	jtop RAM readings at idle, during inference, and peak for each model	Commit to /benchmarks/memory/
Quality scoring sheet	CSV	30 prompts × 4 models, scored 1–3 with brief notes on failures	Commit to /benchmarks/quality_scores.csv

Latency report	PDF or docx	Summary tables, charts, and a recommendation paragraph with reasoning	Share directly with NLP lead for review
Benchmarking script	Python file	Clean, commented, runnable script so results can be reproduced	Commit to /benchmarks/run_benchmark.py

Task 3 MCP Tool Development & Evaluation

Prototype the "arms" of the Dialogue Manager using Model Context Protocol

Why this task matters

The Dialogue Manager in our NLP architecture has two types of outputs: things it says (handled by the TTS engine) and things it does (handled by external tools). For example, answering "What meeting does the MD have at 3pm?" requires the system to call a calendar API, retrieve the answer, and speak it. Without tools, the LLM can only answer from its training data — which means it will hallucinate meeting times.

Model Context Protocol (MCP) is an open standard for defining these tools in a way that LLMs can discover and call them reliably. Think of it as a structured API layer between the LLM and the outside world. Your task is to prototype two MCP tools, evaluate whether MCP is the right choice for our Jetson + LAN server architecture, and write a recommendation.

> Framing: This task is evaluate-and-prototype, not implement-for-production. Your goal is to answer the question: "Does MCP work well enough in our specific architecture to commit to it?" The answer might be yes, might be no, or might be "yes, with these modifications." All three are valuable outcomes.

Detailed task plan

Week	Focus	Daily breakdown
Days	MCP research and setup	Read the official MCP documentation at modelcontextprotocol.io. Understand the server/client model, how tools are defined, and how an LLM calls them. Install the Python MCP SDK. Run the official "hello world" example to confirm your environment works.
Days	Tool 1 — Room availability	Build a mock room availability MCP server. It should expose one tool: get_room_availability(room_name, date) that returns a list of booked time slots. Use a hardcoded JSON fixture as the data source (no real calendar needed yet). Test with an LLM calling the tool and verifying the response.

Days	Tool 2 — Visitor lookup	Build a mock visitor management MCP server. It should expose two tools: get_visitor_info(visitor_name) returning name, company, and expected meeting, and list_todays_visitors() returning all expected visitors for today. Again, use a JSON fixture as the data source.
Days	Architecture evaluation	Test both MCP servers in the Jetson + LAN server hybrid setup. Measure round-trip latency for a tool call. Identify any compatibility issues with ARM64. Write the evaluation report with your recommendation.

MCP tool specifications to implement

Tool 1 — Room availability checker

- **get_room_availability** Tool name:
  - Input: room_name (string), date (string, ISO format)
  - Output: JSON object with fields: room_name, date, booked_slots (list of {start, end, organiser}), available_slots (list of {start, end})
  - Example call: get_room_availability("Boardroom", "2025-04-15")
  - Example response: {"room_name": "Boardroom", "date": "2025-04-15", "booked_slots": [{"start": "09:00", "end": "10:30", "organiser": "Mr. Sharma"}], "available_slots": [{"start": "10:30", "end": "18:00"}]}

Tool 2 — Visitor management lookup

- **get_visitor_info** Tool name:
  - Input: visitor_name (string)
  - Output: JSON object with fields: name, company, expected_arrival, meeting_with, meeting_room, checked_in (boolean)
- **list_todays_visitors** Tool name:
  - Input: none
  - Output: list of visitor objects (same schema as above)

Architecture evaluation checklist

After building both tools, run through each of these evaluation points and record your finding. Your evaluation report must address all of them.

- **ARM64 compatibility:** ARM64 compatibility
  - Does the MCP Python SDK install and run without errors on Jetson Orin? Are there any native dependency issues?
- **Latency overhead:** Latency overhead
  - What is the round-trip time for one tool call (from LLM decision to tool response back in LLM context)? Measure at both ends: Jetson-only and Jetson-to-LAN-server.

- **LLM tool-calling reliability:** LLM tool-calling reliability
  - Does the LLM reliably decide to call the tool when appropriate, and avoid calling it when not needed? Test 20 prompts: 10 that require a tool call and 10 that do not.
- **Error handling:** Error handling
  - What happens when the tool returns an error (e.g., room not found)? Does the LLM handle it gracefully or hallucinate an answer?
- **Streaming compatibility:** Streaming compatibility
  - MCP tool calls pause streaming output while waiting for the result. Is the pause acceptable (< 800ms) in our latency budget?
- **Alternative comparison:** Alternative comparison
  - Compare MCP tool calling against a simple direct function call approach (the LLM outputs JSON, Python parses it and calls the function directly). Is MCP meaningfully better, or is the simpler approach sufficient?

Success metrics — Task 3

Metric	Minimum target	Stretch target	Why it matters
Tool call accuracy	> 80% correct tool invocation on 20-prompt test	> 90%	If the LLM does not call the tool reliably, MCP adds no value
Tool call latency	Round-trip < 800ms on LAN	Round-trip < 400ms	Must fit within the 1.5s total latency budget
ARM64 compatibility	Both tools run on Jetson without errors	Runs with no workarounds needed	Any compatibility issue blocks the entire architecture
Evaluation report quality	All 6 checklist points addressed with data	Includes a clear go/no-go recommendation with reasoning	The NLP lead needs this to make an architecture decision

Deliverables — Task 3

Deliverable	Format	Must include	How to submit
Room availability MCP server	Python package	Working MCP server with get_room_availability tool, fixture data, README to run it	Commit to /mcp/room_availability/
Visitor lookup MCP server	Python package	Working MCP server with get_visitor_info and	Commit to /mcp/visitor_lookup/

		list_todays_visitors tools, fixture data, README	
Integration test script	Python file	20-prompt test harness that calls both tools via an LLM and records accuracy	Commit to /mcp/test_integration.py
Evaluation report	PDF or docx	All 6 checklist points with measured data, latency numbers, ARM64 findings, and a recommendation	Share with NLP lead for architecture decision

Note:

If you are stuck on a technical problem for more than two hours, stop and ask. Do not spend a full day on something that a five-minute conversation could resolve. Send a message describing: what you are trying to do, what you have tried, and what the error or unexpected result is. The more specific your description, the faster the help.

> A note on being stuck: getting stuck is normal and expected. The only thing that is not acceptable is staying stuck silently. If you are confused about the task, ask. If you think the task specification is wrong, say so. If you discover something that changes the recommendation, flag it immediately. Your job is to surface information, not just execute instructions.

Appendix A — Environment Setup

Run the following commands to set up your Python environment. Use Python 3.11 specifically — some dependencies have compatibility issues with 3.12.

```
# Create and activate virtual environmentpython3.11 -m venv nlp-intern-envsource
nlp-intern-env/bin/activate# Core NLP packagespip install openai ollama langchain
langchain-communitypip install spacy && python -m spacy download en_core_web_smpip
install faster-whisper  # quantised Whisper for Jetsonpip install mcp  # Model
Context Protocol SDK# Data and evaluation toolspip install datasets evaluate
scikit-learn pandas# Monitoringpip install jtop  # Jetson resource monitor
```

Appendix B — Key References

- **https://modelcontextprotocol.io/docs** MCP documentation:
- **https://ollama.com/library** Ollama model library:
- **https://github.com/SYSTRAN/faster-whisper** Faster-Whisper (quantised Whisper for edge):
- **https://labelstud.io** Label Studio (annotation tool):
- **https://support.unitree.com/home/en/G1_developer** Unitree G1 SDK docs:

- **https://github.com/rbonghi/jetson_stats** jetson-stats (jtop):